ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Linda Lin**
- **Chinese Name: 林芷瀅**
- **Email: lindalin19720204@gmail.com**
- **Student ID: 113034038**
- **University: National Tsing Hua University**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

594. Different types of articles may use different vocabulary, which can introduce noise when mining patterns.

AI used for this answer: Yes

If yes, how: Grammar polishing of my own draft, find relevant external references

External references (links): Naive Bayes text classification

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer:

If we drop every record with at least one missing value, we will lose many records, which may cause some perspectives to be underrepresented. Instead of dropping these records, we could flag those with missing category labels so that we can identify them easily while retaining their text for other uses.

For example, category_name is not required for counting how often specific words appear. We can still perform word-frequency analysis on documents whose category labels remain unknown.

AI used for this answer: Yes

If yes, how: Grammar polishing of my own draft, find relevant external references

External references (links): fit_transform without category

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

No, there are still small differences between the sampled and original category proportions. For example, the most common category in the original dataset is `soc.religion.christian`, but it changes to `comp.graphics` in the sampled dataset.

In a highly imbalanced dataset, some important features may only appear in categories with fewer records. These categories are less likely to be well represented in random sampling, so we may lose some of their important features.

AI used for this answer: Yes

If yes, how: Grammar polishing and conceptual feedback.

External references (links): Cross-validation iterators with stratification based on class labels

---

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

The number of features increases from 35,488 to 35,788.

The sparsity is expected to decrease because stopwords are common across many documents. Adding them back introduces more non-zero values relative to the total number of matrix elements.

AI used for this answer: Yes

If yes, how: Grammar polishing and conceptual clarification.

External references (links): CountVectorizer

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

Threshold	Number of terms kept (keep == True)
0.0	35482
0.01	5089
0.05	1491

The number of features decreases significantly from 35,482 to 1,491 as the variance threshold increases from 0.0 to 0.05.

Rare words that appear in only a few documents, as well as very common words with similar frequencies across documents, tend to have near-zero variance and are eliminated first.

Near-zero variance means that these features show little variation across documents, making them less useful for distinguishing between documents. Therefore, removing them can reduce dimensionality without significantly affecting model accuracy.

AI used for this answer: Yes

If yes, how: Grammar polishing and conceptual clarification.

External references (links): Removing features with low variance

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

Top terms by |Pearson r| with comp.graphics:

	feature	pearson_r	pearson_p
0	thanks	0.277827	2.782173e-41
1	writes	-0.233682	2.270527e-29
2	keywords	0.232965	3.395522e-29
3	looking	0.213468	1.133072e-24
4	vga	0.211099	3.755895e-24
5	video	0.209458	8.545074e-24
6	people	-0.208871	1.144445e-23
7	card	0.205532	5.934532e-23
8	hi	0.202522	2.553534e-22
9	host	0.199737	9.655856e-22
10	algorithm	0.195873	5.916113e-21

Top terms by |Spearman r| with comp.graphics (among the 1500 highest-variance terms):

	feature	spearman_r	spearman_p
0	graphics	0.521069	2.446204e-157
1	people	-0.319349	1.115413e-54
2	files	0.316719	9.169093e-54
3	image	0.308622	5.266545e-51
4	god	-0.307345	1.408580e-50
5	3d	0.305991	3.975746e-50
6	program	0.291122	2.477587e-45
7	thanks	0.288667	1.440824e-44
8	file	0.283677	4.890639e-43
9	windows	0.276869	5.346111e-41
10	ftp	0.274446	2.750797e-40

The top-ranked features for Pearson are more likely to include generic words, such as "thanks" and "looking", although some are related to graphics. In contrast, Spearman has more domain-specific terms, such as "graphics", "image", and "3d", as well as some negatively correlated words related to other categories.

Pearson measures linear correlation using actual values, while Spearman converts the values into ranks before calculating correlation. Therefore, Spearman focuses on the relative order of values rather than the magnitude of their differences.

Pearson correlation is sensitive to extreme values, which may distort the linear relationship between word counts and categories. Spearman reduces this influence by converting values into ranks before calculating correlation. Therefore, when some documents contain extremely high word counts, Spearman may show a stronger correlation than Pearson.

AI used for this answer: Yes

If yes, how: Grammar polishing and conceptual clarification

External references (links): Pearson Correlation Spearman Rank Correlation

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

```
 TopK frequent patterns were successfully generate
d using FAE algorithm.
Runtime: 0.03381991386413574
K: 100
Actual pattern count: 100
Using kCount: 100 and extracting patterns for cate
gory: alt_atheism
```

[138]:

	Patterns	Support
0	admit	25
1	inimitable	25
2	department	25
3	sea	25
4	authority	24
...	...	...
95	queens sea manhattan	21
96	sea queens	21
97	sank sea manhattan	21
98	sea sank	21
99	sea manhattan	21

100 rows × 2 columns

```
 TopK frequent patterns were successfully generated u
ng FAE algorithm.
Runtime: 0.3288462162017822
K: 500
Actual pattern count: 500
Using kCount: 500 and extracting patterns for categor
alt_atheism
```

[140]:

	Patterns	Support
0	admit	25
1	inimitable	25
2	department	25
3	sea	25
4	authority	24
...	...	...
495	manhattan tektronix blew beaverton	16
496	tektronix blew manhattan	16
497	tektronix blew beaverton	16
498	blew tektronix	16
499	manhattan sank queens bronx blew beaverton	16

500 rows × 2 columns

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup:

```
Maximal Frequent patterns were generated successfu
lly using MaxFp-Growth algorithm
Total No of patterns: 710
Runtime: 0.13474488258361816
Using minimumSupportCount: 9 and extracting patter
ns for category: alt_atheism
```

[141]:

	Patterns	Support
0	server	9
1	spiritual	9
2	introduction	9
3	includes	9
4	address	9
...	...	...
705	takes	24
706	wonder	24
707	couldn	24
708	authority	24
709	admit	25

710 rows × 2 columns

```
Maximal Frequent patterns were generated successfu
lly using MaxFp-Growth algorithm
Total No of patterns: 9547
Runtime: 3.132810354232788
Using minimumSupportCount: 3 and extracting patter
ns for category: alt_atheism
```

[142]:

	Patterns	Support
0	fly	3
1	encompasses	3
2	intentionally	3
3	seven	3
4	owner	3
...	...	...
9542	wonder instance	3
9543	users instance	3
9544	inimitable instance	3
9545	admit instance	3
9546	sea authority	3

9547 rows × 2 columns

The execution time and the number of maximal frequent patterns increased significantly. The former increased from 0.135 to 3.132 seconds, while the latter increased from 710 to 9547.

I think the reason is that more itemsets meet the minimum support threshold, so the search space becomes larger, leading to higher computational cost.

However, the number of maximal patterns is not guaranteed to increase, since a previously maximal pattern may be replaced by a larger frequent superset.

Answer (2) — Top-K k vs MaxFPGrowth minSup:

In Top-K FAE, patterns are ranked by support, and k determines how many top-ranked patterns are retained.

In contrast, minSup specifies the minimum support threshold.

In MaxFPGrowth, only maximal frequent itemsets are retained, so minSup does not directly determine the number of output patterns.

Answer (3) — Maximal vs standard frequent patterns:

Standard frequent pattern mining retains all frequent itemsets, while maximal frequent pattern mining omits their frequent subsets when they are covered by larger frequent patterns. Although this loses the individual support values of the subsets, it reduces redundancy and makes the main frequent patterns easier to observe.

AI used for this answer: Yes

If yes, how: Grammar polishing and conceptual clarification

External references (links): PAMI – FAE PAMI – MaxFPGrowth Frequent Itemset Mining

---

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

1. X_counts has a shape of 2257 x 35788, which means each document is a point in a 35788 dimensional space, but in a scatter plot, each point only has 2 coordinates, and humans can only visualize up to 3 dimensions.
2. we need dimensionality reduction methods like PCA, t-SNE or UMAP to reduce the dimensions before plotting.

AI used for this answer: Yes

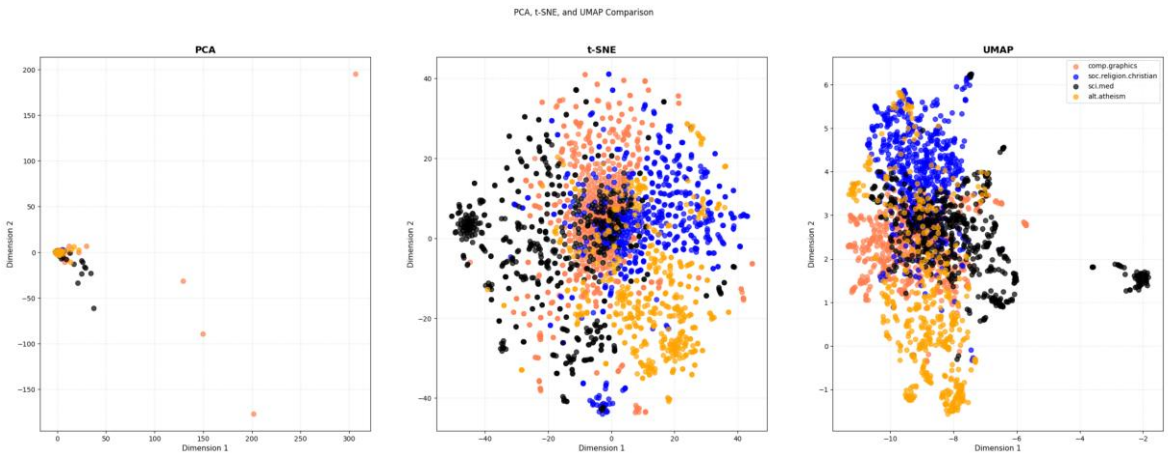
If yes, how: Grammar polishing and conceptual clarification

External references (links): PCA UMAP t-SNE

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.



Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1. t-SNE and UMAP are nonlinear methods, and their plots form separate clusters like islands.
PCA is linear, and most points overlap near the center, with only a few points far away.
But there is still some overlap in the middle of the t-SNE plot.

2. PCA is a linear method that finds directions with the largest variance.
But high variance doesn't always mean better class separation.
For example, if most documents have similar lengths but a few are extremely long, these long documents may dominate PC1, making other points crowded together.In my homework Reddit dataset, the correlation between PC1 and total word count was 0.947.

3. t-SNE mainly preserves local neighbor relationships, meaning which documents are close to each other.
The distances between clusters and their sizes shouldn't be directly interpreted.
Different random seeds can also change the positions of clusters, so their absolute positions and distances aren't reliable.

AI used for this answer: Yes

If yes, how: Grammar polishing and conceptual clarification

External references (links): Use t-SNE Effectively PCA UMAP t-SNE

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1. 4 binary columns are created, one for each category.

2. If we encode four classes as 0, 1, 2, 3, some algorithms might assume there's an order and equal spacing between classes.
   For example, alt.atheism = 0 and soc.religion.christian = 3. If these values are used as features in KNN, their distance would be 3 times the distance between 0 and 1. But these categories don't really have numerical distances.
   Linear regression on these labels would also assume class 2 is between class 1 and 3, which doesn't make sense for unrelated categories.

3. One-hot encoding gives each category its own independent column, so there's no numerical order between classes.
   For example, the Euclidean distance between [1,0,0,0] and [0,1,0,0] is sqrt(2). Any two different one-hot categories have the same distance.
   So the model won't assume one category is closer to another just because of its label number.

AI used for this answer: Yes

If yes, how: Grammar polishing and conceptual clarification

External references (links): scikit-learn LabelBinarizer
